МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
"НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ “МИСИС”

Институт информационных технологий и компьютерных наук

Кафедра инженерной кибернетики

КУРСОВАЯ РАБОТА

по дисциплине “Технологии программирования”

на тему

“Векторно-медианный фильтр для шумоподавления”

Выполнила:
студентка 2 курса
группы БПМ-22-1
Ушакова А.Р.

Проверил:
доцент кафедры
КИК **Полевой Д.В.**

Москва, 2024

Содержание

1. Описание задачи 3
2. Пользовательское описание 4
3. Математическая модель и визуализация работы алгоритма 5
4. Техническая документация 7
4.1 Детали реализации 7
4.2 Инструкция по установке 9
5. Примеры 10
6. Список использованной литературы 11

1. Описание задачи

Целью работы было создание консольного приложения и библиотеки, выполняющих векторно-медианную фильтрацию для шумоподавления изображений. Результат представлен на рисунке 1.1.

(a) **(b)**

Рисунок 1.1 Входное цветное изображение в формате jpg (a) и результат работы алгоритма соответственно (b)

В основные функции CUI-приложения входят:

- Векторно-медианная фильтрация
- Демонстрация работы алгоритма

В основные функции библиотеки входят:

- Функция векторно-медианной фильтрации
- Функция демонстрации алгоритма
- Функция сохранения итогового изображения

2. Пользовательское описание

Библиотека VectorMedianFilter предоставляет функционал для применения векторного медианного фильтра к изображениям. Этот фильтр позволяет сглаживать изображения, удаляя шумы и сохраняя детали.

Для использования функционала библиотеки VectorMedianFilter необходимо создать экземпляр класса ImageProcessor и вызвать метод applyVectorMedianFilter, передав в качестве параметров входное изображение и размер ядра фильтра (окна).

Пример использования:

bash:
./VectorMedianFilter <InputImage> <OutputImage>

Где:
- <InputImage> - путь к входному изображению в формате BGR
- <OutputImage> - путь, по которому будет сохранено выходное изображение после применения векторного медианного фильтра

Пример использования:

bash:
./VectorMedianFilter input.jpg output.jpg

После выполнения программы пользователь получит изображение, обработанное векторно-медианным фильтром, сохраненное по указанному пути <OutputImage>.

3. Математическая модель и визуализация работы алгоритма

Медианный фильтр заменяет каждый пиксель изображения на медиану пикселей в текущей области фильтра, то есть

$$I`(u, v) = median\{I(u+i, v+j\}, (i, j) \in \boldsymbol{R} \qquad (1)$$

Медиана из набора 2n+1 значений A = $\{a_0, \ldots, a_{2n}\}$ может быть определена как центральное значение a_n после упорядочивания (сортировки) A до упорядоченной последовательности, то есть

$$median(a_0, a_1, \ldots, a_{n-1}, a_n, a_{n+1}, \ldots, a_{2n}) = a_n \qquad (2)$$

Где в $a_0, a_1, \ldots, a_{n-1}$ и $a_{n+1}, \ldots, a_{2n}$ содержится n значений и $a_i \leq a_{i+1}$

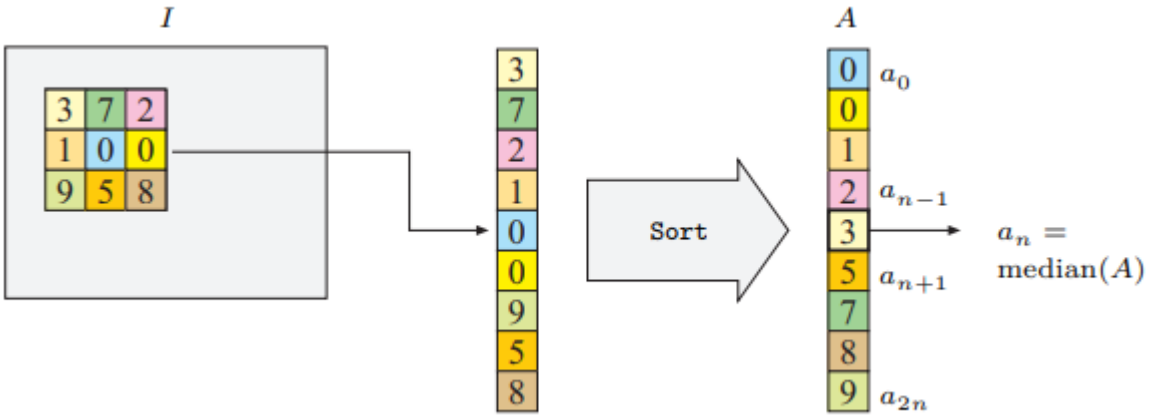

Рисунок 3.1 Пример расчёта медианного фильтра 3×3 (т.е. n = 4)

Уравнение (2) определяет медиану набора значений нечетного размера, и если длина стороны окна нечетна (что обычно и происходит), то количество элементов в области фильтра также нечетно. В этом случае медианный фильтр не создает никаких новых значений пикселей, а лишь выбирает среднее. Однако, если число элементов четное, то медиана отсортированной последовательности A = $\{a_0, \ldots, a_{2n-1}\}$ вычисляется как среднее арифметическое двух соседних центральных значений a_{n-1} и a_n

$$median\{a_0, \ldots, a_{n-1}, a_n, \ldots, a_{2n-1}\} = \frac{a_n + a_{n-1}}{2} \qquad (3)$$

где $a_0, \ldots, a_{n-1}$ состоит из n значений, $a_i \leq a_n$ и $a_n, \ldots, a_{2n-1}$ состоит из n значений, $a_i \geq a_n$

При вычислении среднего обычно вводятся (вычисляются) новые значения пикселей (если длина стороны окна четного размера, что, опять же, нежелательно)

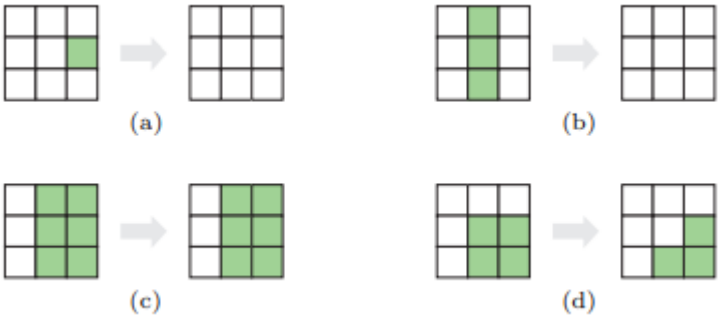

Рисунок 3.2. Влияние медианного фильтра размером 3×3 пикселя на различные структуры 2D-изображений. Удаляются изолированные точки (a) и тонкие линии (b). Край ступеньки остается неизменным (c), в то время как на (d) появляется угол.

4. Техническая документация

4.1 Детали реализации

Использовалась библиотека openCV и её функции

Класс VectorMedianFilter

Класс реализует медианный фильтр для вычисления медианного значения пикселя в заданной окрестности пикселей (окне)

Методы

calculateMedian( )

Vec3b calculateMedian(const vector<Vec3b>& pixels)

Вычисляет медианное значение пикселя из вектора значений пикселей.

Аргументы:

pixels	вектор Vec3b, представляющий значения пикселей в окрестности

Возвращает:
Vec3b - медианное значение пикселя в окрестности

Класс ImageProcessor

Класс предоставляет методы для применения медианного фильтра к входному изображению

Методы

applyVectorMedianFilter( )

Mat applyVectorMedianFilter(const Mat& src, int kSize)

Применяет медианный фильтр к входному изображению

Аргументы:

src	входное изображение в формате Mat (формат CV_8UC3 - изображение в формате BGR)
kSize	размер ядра фильтра/окна (нечетное число)

Возвращает:
Mat - обработанное изображение с примененным медианным фильтром

4.2 Инструкция по установке

Требования для сборки:

- установленная через пакетный менеджер библиотека OpenCV
- CMake версии не ниже 3.21
- язык программирования C++ версии не ниже 17

Вместо [path] нужно прописать путь к менеджеру библиотек (Например: Vcpkg). Для использования последней команды нужно открыть командную строку в режиме администратора, в случае Windows, или прописать перед командой «sudo», в случае Linux/MacOS.

```
1 g i t cl o n e h t t p s : / / https://github.com/ushakova23/misis2023f-22-01-ushakova-a-r
2 cd . / misis2023f-22-01-ushakova-a-r/ushakova23
3 cmake −S . −B build −DCMAKE_TOOLCHAIN_FILE=[ path ]
4 cmake −−build build −−config Release
5 cmake −−install . / build
```

Второй вариант сборки: клонирование git-репозитория в папку с проектом и добавление его, как subdirectory

```
1 cmake_minimum_required (VERSION 3 . 2 1 )
2 project (TEST)
3 add_executable (TEST main . cpp )
4 add_subdirectory (misis2023f-22-01-ushakova-a-r/ushakova23 )
5 target_link_ libraries (TEST median_filter)
```

5. Примеры

Для того чтобы протестировать программу пользователь должен в качестве аргументов командной строки указать адрес входного изображения и адрес выходного изображения.

(a) **(b)**

Рисунок 5.1 Исходное изображение (a) и результат (b)

(a) **(b)**

Рисунок 5.1 Исходное изображение (a) и результат (b)

6. Список использованной литературы

[1] Digital Image Processing An Algorithmic Introduction / Wilhelm Burger, Mark J. Burge — Third Edition. — NY: Springer, Cornell University, 2022 — 943 с.

[2] Проблема подавления шума на изображениях и видео и различные подходы к ее решению [Текст] / Дарья Калинкина, Дмитрий Ватолин — 3-е издание. — 2005 — 30 с.